
Pengembangan Web dengan Django

Pemrograman Berbasis Platform

Ringkasan Materi dan Latihan Soal

Django 5.2 · Python · Arsitektur MTV

Semester Gasal 2026/2027

Contents

1	Arsitektur MTV dan Pemisahan Tanggung Jawab	3
1.1	Siklus satu permintaan	3
1.2	Menyiapkan proyek latihan	3
2	Model dan Migrasi	4
2.1	Mendefinisikan model	4
2.2	Field yang perlu Anda kuasai	5
2.3	UUID sebagai primary key	5
2.4	auto_now_add dibandingkan auto_now	5
2.5	Alur migrasi	6
2.6	Memeriksa model lewat shell	6
2.7	Drill 1: Model dan Migrasi	6
3	Routing URL	7
3.1	Struktur dua lapis	7
3.2	Nama rute dan mengapa nama tersebut wajib	7
3.3	Konverter jalur	8
3.4	Drill 2: Routing dan Nama Rute	8
4	View, Method HTTP, dan Pola Redirect	8
4.1	Anatomi view fungsi	8
4.2	Memilih method HTTP yang tepat	9
4.3	Membatasi method dengan dekorator	9
4.4	Pola Post/Redirect/Get	10
4.5	Drill 3: Aksi POST, Penolakan Method, dan Redirect	10
5	Form, Validasi, dan CSRF	11
5.1	ModelForm	11
5.2	Pemrosesan manual	11
5.3	CSRF	12
5.4	Drill 4: ModelForm, Validasi, dan Post/Redirect/Get	12
6	Template	13
6.1	Pewarisan template	13
6.2	Tombol aksi harus berupa form POST	14
6.3	Sintaks yang sering dipakai	14
6.4	Drill 5: Template, Keadaan Kosong, dan Tombol Aksi	14
7	Kueri, Pengambilan Acak, dan Edge Case	15
7.1	Operasi QuerySet yang penting	15
7.2	Mengambil satu baris secara acak	15
7.3	Daftar edge case yang wajib diperiksa	15
7.4	Drill 6: Kueri, Penyortan, dan Edge Case	16
8	Serialisasi JSON dan XML	16
8.1	Serializer bawaan Django	16
8.2	Header Content-Type bukan sekadar pelengkap	16
8.3	JsonResponse untuk struktur buatan sendiri	17
8.4	Menyaring lewat query parameter	17
8.5	Drill 7: Endpoint Serialisasi	17

9	Pengujian Otomatis	18
9.1	Struktur berkas uji	18
9.2	Kerangka pengujian	18
9.3	Asersi yang perlu dihafal	20
9.4	Basis data uji bersifat terisolasi	20
9.5	Mengukur cakupan	20
9.6	Drill 8: Menulis Pengujian dan Menaikkan Cakupan	21
10	Kualitas Kode	21
10.1	Menjalankan ruff	21
10.2	Kebiasaan yang menjaga kualitas kode	21
10.3	Drill 9: Menemukan Cacat pada Kode Orang Lain	22
11	Daftar Kesalahan yang Paling Sering Terjadi	23
12	Latihan Terpadu: 100 Menit	24
13	Soal Konseptual	25
14	Kunci Jawaban dan Pembahasan	26
14.1	Kunci Drill 1	26
14.2	Kunci Drill 2	26
14.3	Kunci Drill 3	27
14.4	Kunci Drill 4	27
14.5	Kunci Drill 5	28
14.6	Kunci Drill 6	29
14.7	Kunci Drill 7	29
14.8	Kunci Drill 8	30
14.9	Kunci Drill 9	31
14.10	Kunci Soal Terpadu	31
14.11	Kunci Soal Konseptual	32

1 Arsitektur MTV dan Pemisahan Tanggung Jawab

Django menyebut arsitekturnya Model-Template-View. Penamaan tersebut sering membingungkan karena *view* pada Django berperan sebagai *controller* pada istilah MVC, sedangkan *template* berperan sebagai *view*. Hal yang benar-benar penting bukan hafalan istilahnya, melainkan apakah setiap potongan kode diletakkan pada lapisan yang benar.

Lapisan	Berkas	Tanggung jawab
Model	<code>models.py</code>	Struktur data, aturan validasi tingkat data, perilaku yang melekat pada satu entitas
View	<code>views.py</code>	Menerima <code>HttpRequest</code> , memanggil model, memutuskan respons, tidak menyusun HTML secara manual
Template	<code>templates/</code>	Menampilkan data yang sudah disiapkan view, hanya berisi logika presentasi ringan
URLconf	<code>urls.py</code>	Memetakan pola URL ke view dan memberi nama pada setiap rute
Form	<code>forms.py</code>	Validasi dan pembersihan masukan pengguna

Pelanggaran pemisahan yang paling sering muncul adalah menyusun string HTML di dalam `views.py`, menaruh kueri basis data yang rumit di dalam template, serta menulis logika validasi yang sama berulang kali di beberapa view alih-alih memindahkannya ke `forms.py` atau ke model.

1.1 Siklus satu permintaan

Memahami urutan berikut membantu Anda mendiagnosis galat dengan cepat, karena setiap jenis galat muncul pada tahap yang berbeda.

1. Peramban mengirim permintaan ke suatu URL.
2. Django mencocokkan URL dengan pola di `urls.py`. Kegagalan pada tahap ini menghasilkan galat 404 beserta daftar pola yang dicoba.
3. View yang cocok dipanggil dengan objek `request` dan argumen dari URL. Kegagalan di sini menghasilkan galat 500 beserta *traceback*.
4. View mengakses model, menyiapkan *context*, lalu memanggil `render` atau mengembalikan `HttpResponse`.
5. Template dirender menjadi HTML. Kesalahan nama variabel di template tidak menimbulkan galat, melainkan menghasilkan string kosong. Sifat ini membuat kesalahan template sulit terdeteksi.

Membaca halaman galat Django

Saat `DEBUG=True`, halaman galat menampilkan baris kode penyebab beserta nilai variabel lokal. Membaca dua baris teratas *traceback* dan nama *exception* hampir selalu cukup untuk menemukan penyebabnya. Galat 404 menampilkan daftar pola URL yang dicoba, yang langsung memperlihatkan apakah rute Anda salah tulis atau belum terdaftar.

1.2 Menyiapkan proyek latihan

Setiap bab materi mulai dari Bab 2 diakhiri satu drill pendek yang dirancang selesai dalam 10 sampai 20 menit. Kesembilan drill tersebut saling menyambung dan membangun satu aplikasi kecil yang sama, sehingga hasil setiap drill dipakai kembali pada drill berikutnya. Bab 12 menggabungkan seluruhnya dalam satu sesi bertimer, Bab 13 menguji pemahaman yang tidak terlihat dari kode yang kebetulan berjalan, dan kunci jawaban berada pada bab terakhir.

Seluruh drill memakai domain yang berbeda dari contoh di dalam materi agar Anda melatih polanya, bukan menghafal jawabannya. Titik awalnya adalah satu repositori kerangka yang sudah berjalan tetapi sengaja dikosongkan. Kloning repositori tersebut sebelum masuk ke Bab 2.

```
git clone https://github.com/aldofahrezy/latihan-quiz-1.git latihan-django
cd latihan-django

python -m venv env
source env/bin/activate      # macOS atau Linux
env\Scripts\activate         # Windows

pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Buka <http://127.0.0.1:8000/>. Jika halaman beranda tampil, lingkungan Anda sudah siap. Kerangka tersebut sudah memuat aplikasi main yang terdaftar pada `INSTALLED_APPS`, direktori `templates/` dan `static/` yang sudah dikenali, satu rute beranda bernama `show_main`, berkas `base.html` yang siap diwarisi, serta `main/tests/` lengkap dengan `__init__.py`. Berkas `main/models.py` dan `main/forms.py` sengaja dibiarkan kosong karena keduanya baru diisi pada drill.

Buat cabang kerja sendiri agar riwayat latihan Anda terpisah dari kerangkanya.

```
git switch -c latihan-saya
```

Drill saling menyangkut

Kesembilan drill membangun satu aplikasi yang sama secara bertahap, sehingga melewati satu drill membuat drill berikutnya kehilangan kode yang diandalkannya. Jika Anda terpaksa melompati satu bagian, salin lebih dahulu kunci drill tersebut agar titik awal Anda setara.

2 Model dan Migrasi

2.1 Mendefinisikan model

Model adalah kelas Python yang diturunkan dari `models.Model`. Setiap atribut kelas yang bertipe `Field` menjadi kolom pada tabel basis data.

```
import uuid
from django.db import models

class Note(models.Model):
    class Category(models.TextChoices):
        GENERAL = "general", "General"
        URGENT = "urgent", "Urgent"

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    title = models.CharField(max_length=120)
    body = models.TextField()
    category = models.CharField(
        max_length=20, choices=Category.choices, default=Category.GENERAL
    )
    is_pinned = models.BooleanField(default=False)
    view_count = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

```
class Meta:
    ordering = ["-created_at"]

def __str__(self):
    return self.title
```

2.2 Field yang perlu Anda kuasai

Field	Keterangan
CharField	Teks pendek. Wajib menyertakan <code>max_length</code> .
TextField	Teks panjang tanpa batas eksplisit. Tidak memerlukan <code>max_length</code> .
BooleanField	Nilai benar atau salah. Beri <code>default</code> agar migrasi tidak meminta nilai.
IntegerField, PositiveIntegerField	Bilangan bulat. Varian <i>positive</i> menolak nilai negatif.
DateTimeField	Waktu. Lihat pembahasan <code>auto_now_add</code> di bawah.
UUIDField	Pengenal unik 128 bit. Umum dipakai sebagai primary key.
ForeignKey	Relasi banyak ke satu. Wajib menyertakan <code>on_delete</code> .

2.3 UUID sebagai primary key

Jika spesifikasi meminta pengenal berupa UUID, tiga argumen berikut harus muncul bersamaan.

```
id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
```

Argumen `primary_key=True` menggantikan kolom `id` otomatis bertipe integer. Argumen `default=uuid.uuid4` menyerahkan pembuatan nilai kepada Django saat objek dibuat. Perhatikan bahwa yang dituliskan adalah `uuid.uuid4` tanpa tanda kurung, karena Django memerlukan fungsinya, bukan hasil pemanggilannya. Menulis `uuid.uuid4()` menyebabkan seluruh baris memakai UUID yang sama dan melanggar keunikan primary key. Argumen `editable=False` menyembunyikan kolom tersebut dari form.

UUID di dalam URL

Rute yang menerima UUID harus memakai konverter `uuid`, bukan `int` atau `str`. Pola `"item/<int:pk>/"` tidak akan pernah cocok dengan primary key UUID sehingga menghasilkan 404 yang membingungkan. Pola yang benar adalah `"item/<uuid:pk>/"`.

2.4 auto_now_add dibandingkan auto_now

Kedua argumen tersebut mudah tertukar, padahal maknanya berlawanan.

Argumen	Waktu pengisian	Dipakai untuk
<code>auto_now_add=True</code>	Hanya sekali, saat objek pertama kali disimpan	Stempel waktu pembuatan
<code>auto_now=True</code>	Setiap kali <code>save()</code> dipanggil	Stempel waktu pembaruan terakhir
<code>default=timezone.now</code>	Saat objek dibuat, tetapi masih dapat diubah	Waktu yang boleh disunting pengguna

Field dengan `auto_now_add` atau `auto_now` bersifat non-editable. Field tersebut tidak akan muncul pada `ModelForm` dan nilainya tidak dapat diubah lewat form, meskipun Anda mencantulkannya pada `Meta.fields`.

2.5 Alur migrasi

```
python manage.py makemigrations    # menulis skrip migrasi dari perubahan model
python manage.py migrate          # menerapkan skrip ke basis data
python manage.py showmigrations    # memeriksa migrasi mana yang sudah diterapkan
```

Berkas migrasi yang dihasilkan berada di `main/migrations/` dan wajib ikut di-*commit*. Siapa pun yang menjalankan ulang proyek Anda dari salinan repositori memerlukan berkas tersebut, karena tanpa migrasi tabelnya tidak akan pernah terbentuk.

Menambah field wajib pada tabel yang sudah berisi data

Jika Anda menambahkan field tanpa default ke model yang tabelnya sudah berisi baris, `makemigrations` akan berhenti dan meminta nilai awal secara interaktif. Pada situasi yang serba cepat, jeda tak terduga semacam itu mudah memicu kesalahan. Cegah dengan selalu memberi default pada field baru, atau dengan `null=True` bila field memang boleh kosong.

2.6 Memeriksa model lewat shell

Setelah migrasi berhasil, verifikasi model Anda sebelum menulis view. Pemeriksaan ini memisahkan kesalahan model dari kesalahan view.

```
# python manage.py shell
from main.models import Note

n = Note.objects.create(title="Halo", body="Isi catatan")
print(n.id, n.created_at, n.is_pinned)    # UUID, waktu terisi, False
print(Note.objects.count())               # 1
```

2.7 Drill 1: Model dan Migrasi

Drill 1 (15 menit)

Berkas `main/models.py` pada kerangka masih kosong. Isi berkas tersebut dengan model `Bookmark` sesuai ketentuan berikut.

- `id`: primary key bertipe `UUID` yang terisi otomatis dan tidak dapat disunting.
- `label`: teks pendek maksimal 100 karakter.
- `url`: alamat tautan, gunakan field yang tepat agar Django memvalidasi formatnya.
- `priority`: bilangan bulat tak negatif dengan nilai bawaan 0.
- `is_archived`: boolean dengan nilai bawaan `False`.
- `created_at`: waktu pembuatan yang terisi otomatis sekali saja.
- `last_opened_at`: waktu yang boleh kosong dan boleh bernilai `null`.

Tambahkan pengurutan bawaan berdasarkan `priority` menurun kemudian `created_at` menurun, serta `__str__` yang mengembalikan `label`. Jalankan migrasi, lalu buktikan lewat `manage.py shell` bahwa nilai bawaan dan pengurutannya bekerja.

3 Routing URL

3.1 Struktur dua lapis

Proyek Django memiliki satu `urls.py` di tingkat proyek dan biasanya satu `urls.py` per aplikasi. Lapisan proyek mendelegasikan sebagian jalur ke aplikasi memakai `include`.

```
# proyek/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("main.urls")),
]
```

```
# main/urls.py
from django.urls import path
from main.views import (
    show_main,
    show_notes,
    toggle_pinned_note,
    delete_note,
    notes_data,
)

app_name = "main"

urlpatterns = [
    path("", show_main, name="show_main"),
    path("notes/", show_notes, name="show_notes"),
    path("notes/<uuid:note_id>/pin/", toggle_pinned_note, name="toggle_pinned_note"),
    path("notes/<uuid:note_id>/delete/", delete_note, name="delete_note"),
    path("notes/data/", notes_data, name="notes_data"),
]
```

3.2 Nama rute dan mengapa nama tersebut wajib

Argumen `name` memungkinkan Anda menyebut rute tanpa menuliskan URL secara harfiah. Template dan kode Python memanggil rute yang sama lewat namanya.

```
reverse("main:show_notes")          # di dalam views.py atau tests.py
{% url 'main:show_notes' %}         # di dalam template
```

Jika spesifikasi menyebutkan nama rute tertentu, nama tersebut adalah bagian dari kontrak dan harus ditulis persis. Pengujian otomatis lazim memanggil `reverse` dengan nama itu sehingga satu huruf yang keliru membuat seluruh rangkaian pengujian pada bagian tersebut gagal sekaligus.

Urutan pola dan garis miring penutup

Django mencocokkan pola dari atas ke bawah dan berhenti pada kecocokan pertama. Pola `"notes/<str:slug>/"` yang diletakkan di atas `"notes/data/"` akan menelan permintaan ke `/notes/data/` karena `data` cocok sebagai `slug`. Letakkan pola yang lebih spesifik di atas pola yang lebih umum.

Selain itu Django membedakan `/notes` dan `/notes/`. Bila spesifikasi menuliskan garis miring penutup, tuliskan persis seperti itu.

3.3 Konverter jalur

Konverter	Mencocokkan	Tipe argumen di view
int	Satu atau lebih digit	int
str	Teks apa pun kecuali garis miring	str
slug	Huruf, angka, tanda hubung, garis bawah	str
uuid	UUID berformat baku	uuid.UUID
path	Teks apa pun termasuk garis miring	str

3.4 Drill 2: Routing dan Nama Rute

Drill 2 (10 menit)

Kerangka sudah menyediakan `main/urls.py` dengan `app_name = "main"` dan satu rute beranda bernama `show_main`. Tambahkan empat rute berikut di bawahnya.

Pola	Nama
<code>/bookmarks/</code>	<code>show_bookmarks</code>
<code>/bookmarks/<id>/archive/</code>	<code>toggle_archive</code>
<code>/bookmarks/<id>/delete/</code>	<code>delete_bookmark</code>
<code>/bookmarks/data/</code>	<code>bookmarks_data</code>

Gunakan konverter yang sesuai untuk primary key UUID. View-nya belum ada, jadi untuk sementara arahkan keempat rute tersebut ke fungsi placeholder yang mengembalikan `HttpResponse` kosong. Pastikan `python manage.py check` tetap bersih. Kemudian jawab dua pertanyaan berikut secara tertulis.

1. Jika pola `"bookmarks/<str:tag>/"` ditambahkan tepat di atas `"bookmarks/data/"`, apa yang terjadi saat pengguna membuka `/bookmarks/data/` dan mengapa?
2. Tuliskan ekspresi `reverse` dan tag `{% url %}` untuk rute `toggle_archive` dengan argumen sebuah objek bernama `b`.

4 View, Method HTTP, dan Pola Redirect

4.1 Anatomi view fungsi

```
from django.shortcuts import render

def show_notes(request):
    notes = Note.objects.all()
    context = {"notes": notes}
    return render(request, "notes.html", context)
```

View menerima `HttpRequest` sebagai argumen pertama dan wajib mengembalikan `HttpResponse`. Fungsi `render` adalah pintasan yang memuat template, mengisinya dengan `context`, lalu membungkus hasilnya menjadi `HttpResponse`.

4.2 Memilih method HTTP yang tepat

Pemilihan method bukan formalitas. GET bersifat aman dan idempoten sehingga peramban, *prefetcher*, dan *crawler* boleh memanggилnya kapan saja tanpa izin pengguna. Karena itu operasi yang mengubah atau menghapus data tidak boleh dapat dipicu lewat GET. Tautan penghapusan berbasis GET dapat terpicu hanya karena peramban melakukan *prefetch*, dan data hilang tanpa ada yang mengkliknya.

Method	Dipakai untuk	Sifat
GET	Membaca dan menampilkan	Aman, idempoten, boleh disinggahkan
POST	Membuat, mengubah, menghapus	Tidak aman, memerlukan token CSRF

4.3 Membatasi method dengan dekorator

```
from django.shortcuts import get_object_or_404, redirect
from django.views.decorators.http import require_POST, require_GET, require_http_methods

@require_POST
def toggle_pinned_note(request, note_id):
    note = get_object_or_404(Note, pk=note_id)
    note.is_pinned = not note.is_pinned
    note.save(update_fields=["is_pinned", "updated_at"])
    return redirect("main:show_notes")

@require_POST
def delete_note(request, note_id):
    note = get_object_or_404(Note, pk=note_id)
    note.delete()
    return redirect("main:show_notes")
```

Dekorator `@require_POST` membuat Django mengembalikan status 405 Method Not Allowed untuk permintaan selain POST. Terdapat pula cara manual yang memberi hasil setara, tetapi lebih panjang dan lebih mudah salah.

```
from django.http import HttpResponseNotAllowed

def delete_note(request, note_id):
    if request.method != "POST":
        return HttpResponseNotAllowed(["POST"])
    ...
```

get_object_or_404 dibandingkan get

Pemanggilan `Note.objects.get(pk=...)` melemparkan `Note.DoesNotExist` untuk id yang tidak ada, dan *exception* tersebut menjadi galat 500. Fungsi `get_object_or_404` mengubahnya menjadi 404 yang merupakan respons yang benar untuk sumber daya yang tidak ditemukan. Perbedaan tersebut penting karena 404 memberi tahu klien bahwa permintaannya yang keliru, sedangkan 500 menyatakan aplikasi Andalah yang rusak.

4.4 Pola Post/Redirect/Get

Setelah memproses POST yang mengubah data, view harus mengembalikan redirect, bukan merender halaman secara langsung. Alasannya adalah bila view merender halaman langsung dari POST, menekan tombol muat ulang pada peramban akan mengirim ulang POST tersebut. Data yang sama tersimpan dua kali atau penghapusan terjadi berulang. Redirect mengubah permintaan berikutnya menjadi GET biasa sehingga muat ulang menjadi aman.

```
def show_notes(request):
    if request.method == "POST":
        form = NoteForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("main:show_notes")  # PRG: redirect, bukan render
    else:
        form = NoteForm()

    context = {"form": form, "notes": Note.objects.all()}
    return render(request, "notes.html", context)
```

Perhatikan bahwa ketika form tidak valid, view sengaja merender ulang halaman alih-alih melakukan redirect. Perilaku tersebut mempertahankan masukan pengguna beserta pesan galatnya, sedangkan redirect akan menghapus keduanya.

4.5 Drill 3: Aksi POST, Penolakan Method, dan Redirect

Drill 3 (20 menit)

Pada tahap ini form belum dibahas, jadi isi basis data lebih dahulu lewat `manage.py shell` agar ada data yang dapat ditindaklanjuti. Kemudian implementasikan tiga view berikut.

1. `show_bookmarks`: menangani GET dan menampilkan seluruh bookmark. Buat `main/templates/main/bookmarks.html` yang mewarisi `base.html`. Untuk sementara isinya boleh sesederhana satu daftar tak berurut.
2. `toggle_archive`: hanya menerima POST, membalik nilai `is_archived`, lalu redirect kembali ke `show_bookmarks`.
3. `delete_bookmark`: hanya menerima POST, menghapus baris, lalu redirect kembali ke `show_bookmarks`.

Ganti fungsi placeholder dari Drill 2 dengan implementasi sebenarnya. Gunakan `@require_POST` dan `get_object_or_404`, serta lakukan redirect memakai nama rute, bukan URL literal. Agar tombolnya dapat ditekan, bungkus setiap tombol dengan `<form method="post">` dan sertakan `{% csrf_token %}` di dalamnya. Alasan tag tersebut diperlukan dibahas pada bab berikutnya.

Setelah selesai, verifikasi ketiga perilaku berikut.

- Menempelkan `/bookmarks/<id>/delete/` langsung ke address bar menghasilkan 405 dan data tetap utuh.
- Mengirim POST ke rute arsip dengan UUID acak yang tidak ada di basis data menghasilkan 404, bukan 500. Perhatikan bahwa pemeriksaan ini tidak dapat dilakukan lewat address bar, karena `@require_POST` diperiksa lebih dahulu sehingga GET ke id yang tidak ada tetap menjawab 405. Kirim POST-nya lewat `manage.py shell`.
- Setelah tombol arsip ditekan, alamat pada address bar kembali menjadi `/bookmarks/`. Perubahan alamat itulah bukti bahwa redirect benar-benar terjadi.

```
# python manage.py shell
import uuid
from django.test import Client
from django.urls import reverse

c = Client()
url = reverse("main:toggle_archive", args=[uuid.uuid4()])
print(c.post(url).status_code)    # 404
print(c.get(url).status_code)    # 405
```

5 Form, Validasi, dan CSRF

5.1 ModelForm

ModelForm menurunkan definisi field, validasi, dan penyimpanan langsung dari model. Pendekatan tersebut lebih ringkas dan lebih tahan terhadap perubahan model dibandingkan pemrosesan manual.

```
# main/forms.py
from django import forms
from main.models import Note

class NoteForm(forms.ModelForm):
    class Meta:
        model = Note
        fields = ["title", "body", "category"]
        widgets = {
            "body": forms.Textarea(attrs={"rows": 3, "placeholder": "Tulis catatan"}),
        }

    def clean_title(self):
        title = self.cleaned_data["title"].strip()
        if len(title) < 3:
            raise forms.ValidationError("Judul minimal tiga karakter.")
        return title
```

Daftar fields sebaiknya ditulis eksplisit. Menulis fields = "__all__" memang lebih cepat, tetapi setiap field baru pada model otomatis ikut terbuka untuk disunting pengguna, termasuk field yang seharusnya hanya diatur sistem.

5.2 Pemrosesan manual

Bila Anda memilih tidak memakai ModelForm, validasi tetap wajib dilakukan. Data dari request.POST selalu bertipe string dan selalu berasal dari luar sehingga tidak boleh dipercaya begitu saja.

```
def show_notes(request):
    error = None
    if request.method == "POST":
        title = request.POST.get("title", "").strip()
        body = request.POST.get("body", "").strip()
        if not title or not body:
            error = "Judul dan isi tidak boleh kosong."
        else:
            Note.objects.create(title=title, body=body)
            return redirect("main:show_notes")
```

```
context = {"notes": Note.objects.all(), "error": error}
return render(request, "notes.html", context)
```

Penggunaan `request.POST.get("title", "")` lebih aman daripada `request.POST["title"]`, karena akses dengan tanda kurung siku melemparkan `MultiValueDictKeyError` dan menghasilkan galat 500 ketika field tersebut tidak dikirim.

5.3 CSRF

Cross-Site Request Forgery adalah serangan yang membuat peramban korban mengirim permintaan ke situs tempat korban sudah masuk, atas perintah situs lain. Django menangkalnya dengan menuntut setiap permintaan POST menyertakan token rahasia yang hanya diketahui halaman asli. Tag `{% csrf_token %}` menyisipkan token tersebut sebagai input tersembunyi.

```
<form method="post" action="{% url 'main:show_notes' %}">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Tambah catatan</button>
</form>
```

Gejala CSRF yang hilang

Form POST tanpa `{% csrf_token %}` menghasilkan halaman 403 Forbidden bertuliskan CSRF verification failed. Galat tersebut muncul saat menekan tombol kirim, bukan saat membuka halaman. Karena itu selalu uji tombol kirim Anda, bukan hanya tampilan halamannya. Perhatikan pula bahwa tag tersebut hanya berlaku untuk POST dan tidak diperlukan pada form GET seperti kotak pencarian.

5.4 Drill 4: ModelForm, Validasi, dan Post/Redirect/Get

Drill 4 (20 menit)

Lanjutkan pekerjaan Drill 3 dengan menambahkan jalur penambahan data.

1. Buat `BookmarkForm` sebagai `ModelForm` yang hanya membuka field `label`, `url`, dan `priority`. Tambahkan dua validasi khusus: `label` dipangkas spasinya dan wajib berisi minimal tiga karakter, sedangkan `priority` tidak boleh lebih dari 10.
2. Perluas `show_bookmarks` agar juga menangani POST. Bila form valid, simpan lalu redirect ke rute yang sama. Bila tidak valid, render ulang halaman beserta pesan galatnya tanpa redirect.
3. Render form tersebut pada template beserta `{% csrf_token %}` dan tombol kirim.

Setelah selesai, lakukan tiga pemeriksaan berikut.

- Kirim label sepanjang dua karakter. Data tidak boleh bertambah, pesan galat harus tampil, dan masukan yang sudah diketik tidak boleh hilang.
- Tambahkan satu bookmark yang valid, lalu tekan tombol muat ulang peramban. Tidak boleh ada data ganda.
- Hapus `{% csrf_token %}`, muat ulang halaman, lalu tekan tombol kirim. Catat kode status beserta pesan yang muncul, kemudian kembalikan tag tersebut.

6 Template

6.1 Pewarisan template

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Situs Saya{% endblock %}</title>
</head>
<body>
  <header><nav><a href="{% url 'main:show_main' %}">Beranda</a></nav></header>
  <main>{% block content %}{% endblock %}</main>
  <footer><p>&copy; 2026</p></footer>
</body>
</html>
```

```
<!-- main/templates/notes.html -->
{% extends 'base.html' %}

{% block title %}Daftar Catatan{% endblock %}

{% block content %}
<h1>Daftar Catatan</h1>

<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Tambah</button>
</form>

<ul>
  {% for note in notes %}
    <li>
      <article>
        <h2>{{ note.title }}</h2>
        <p>{{ note.body }}</p>
        <time datetime="{% note.created_at|date:'c' %}">
          {{ note.created_at|date:"d M Y H:i" }}
        </time>
      </article>

      <form method="post" action="{% url 'main:toggle_pinned_note' note.id %}">
        {% csrf_token %}
        <button type="submit">{% if note.is_pinned %}Lepas Pin{% else %}Pin{% endif %}</button>
      </form>

      <form method="post" action="{% url 'main:delete_note' note.id %}">
        {% csrf_token %}
        <button type="submit">Hapus</button>
      </form>
    </li>
    {% empty %}
    <li>Belum ada catatan.</li>
  {% endfor %}
</ul>
{% endblock %}
```

Klausula `{% empty %}` di dalam `{% for %}` menangani keadaan daftar kosong tanpa memerlukan

{% if %} tambahkan. Klausa tersebut adalah cara idiomatik Django untuk menampilkan pesan cadangan ketika data belum ada.

6.2 Tombol aksi harus berupa form POST

Perhatikan bahwa tombol Pin dan Hapus pada contoh di atas masing-masing dibungkus `<form method="post">` lengkap dengan token CSRF, bukan ditulis sebagai `<a href="...">`. Tautan biasa menghasilkan permintaan GET yang akan ditolak oleh `@require_POST`. Kesalahan tersebut menghasilkan gejala khas berupa halaman 405 setiap kali tombol ditekan.

6.3 Sintaks yang sering dipakai

Sintaks	Fungsi
<code>{{ variabel }}</code>	Menampilkan nilai
<code>{{ nilai default:"- " }}</code>	Nilai cadangan bila kosong
<code>{{ teks truncatewords:20 }}</code>	Memotong teks
<code>{{ waktu date:"d M Y" }}</code>	Memformat tanggal
<code>{% if %} {% elif %} {% else %}</code>	Percabangan
<code>{% endif %}</code>	
<code>{% for x in xs %} {% empty %}</code>	Perulangan beserta keadaan kosong
<code>{% endfor %}</code>	
<code>{% url 'app:nama' arg %}</code>	Membangun URL dari nama rute
<code>{% csrf_token %}</code>	Token keamanan untuk form POST

Kegagalan template bersifat senyap

Salah tulis nama variabel di template tidak memunculkan galat. Django merender string kosong dan halaman tampak normal tetapi datanya hilang. Bila sebuah nilai tidak muncul, periksa terlebih dahulu ejaan kunci pada *context* view dan ejaan variabel pada template. Kunci "note" pada view tidak akan terbaca oleh `{{ notes }}` di template.

6.4 Drill 5: Template, Keadaan Kosong, dan Tombol Aksi

Drill 5 (15 menit)

Kerangka sudah menyediakan `templates/base.html` beserta kelas CSS untuk daftar dan tombol aksi. Rapihan `main/templates/main/bookmarks.html` yang Anda buat pada Drill 3 sehingga memenuhi ketentuan berikut.

- Menampilkan seluruh bookmark memakai elemen HTML semantik.
- Menampilkan pesan *Belum ada bookmark tersimpan.* ketika daftar kosong, memakai klausa `{% empty %}`.
- Setiap baris memiliki tombol arsip yang teksnya *Kembalikan* ketika `is_archived` bernilai `True` dan *Arsipkan* ketika bernilai `False`.
- Setiap baris memiliki tombol hapus.
- Kedua tombol dikirim lewat form POST lengkap dengan token CSRF.
- Waktu pembuatan ditampilkan dengan format *dd Mon yyyy* di dalam elemen `<time>`.

Sebagai pemeriksaan tambahan, ubah sengaja nama variabel pada template menjadi salah, misalnya `{{ bookmark.label1 }}`. Amati bahwa halaman tetap tampil tanpa galat dan jelaskan mengapa perilaku tersebut berbahaya.

7 Kueri, Pengambilan Acak, dan Edge Case

7.1 Operasi QuerySet yang penting

```
Note.objects.all()           # seluruh baris
Note.objects.filter(is_pinned=False) # penyaringan
Note.objects.exclude(category="urgent") # kebalikan filter
Note.objects.order_by("-created_at") # pengurutan menurun
Note.objects.count()         # jumlah baris
Note.objects.exists()        # lebih murah daripada count() > 0
Note.objects.first()         # objek pertama atau None
Note.objects.filter(title__icontains="rapat") # pencarian tanpa peduli huruf besar
```

QuerySet bersifat malas. Kueri baru dikirim ke basis data saat hasilnya benar-benar dipakai, misalnya ketika diiterasi, diubah menjadi list, atau dihitung panjangnya. Sifat tersebut memungkinkan Anda merangkai beberapa filter tanpa biaya tambahan.

7.2 Mengambil satu baris secara acak

Terdapat tiga pendekatan yang sama-sama benar. Perbedaannya terletak pada keterbacaan dan biaya.

```
import random

# Pendekatan 1: paling ringkas. Mengembalikan None bila tidak ada baris.
note = Note.objects.filter(is_pinned=False).order_by("?").first()

# Pendekatan 2: eksplisit dan mudah dibaca.
candidates = list(Note.objects.filter(is_pinned=False))
note = random.choice(candidates) if candidates else None

# Pendekatan 3: menghindari memuat seluruh tabel ke memori.
qs = Note.objects.filter(is_pinned=False)
total = qs.count()
note = qs[random.randrange(total)] if total else None
```

Pendekatan pertama adalah yang terpendek, tetapi `order_by("?")` memaksa basis data mengacak seluruh tabel sehingga menjadi lambat pada tabel besar. Pada tabel berukuran kecil, perbedaan tersebut tidak terasa. Mencatat pertimbangannya dalam komentar singkat membantu pembaca berikutnya memahami mengapa pendekatan itu dipilih.

Mengapa `.first()` lebih aman daripada `[0]`

] Ekspresi `qs[0]` melemparkan `IndexError` pada QuerySet kosong, dan *exception* tersebut menjadi galat 500. Sebaliknya `qs.first()` mengembalikan `None`. Nilai `None` dapat ditangani template dengan `{% if %}` sehingga halaman tetap tampil rapi. Menangani keadaan kosong tanpa galat 500 adalah syarat minimum agar halaman tetap dapat dibuka pada basis data yang baru dibuat.

7.3 Daftar edge case yang wajib diperiksa

Sebelum menganggap sebuah fitur selesai, periksa keempat keadaan berikut secara manual di peramban.

1. Basis data benar-benar kosong. Halaman harus menampilkan pesan cadangan, bukan halaman galat.
2. Basis data berisi data, tetapi tidak ada satu pun yang lolos filter. Keadaan ini berbeda dari keadaan pertama dan sering terlewat.

3. Identitas objek yang diminta tidak ada. Hasil yang benar adalah 404, bukan 500.
4. Method HTTP yang salah dikirim ke rute yang membatasi method. Hasil yang benar adalah 405.

7.4 Drill 6: Kueri, Penyorotan, dan Edge Case

Drill 6 (20 menit)

Kerangka menyediakan halaman beranda `show_main` beserta `main/templates/main/home.html` yang sudah memuat satu bagian bertajuk Sorotan dengan isi sementara. Gantilah isi bagian tersebut dengan satu bookmark sorotan yang dipilih memakai aturan berjenjang berikut.

1. Jika terdapat bookmark tidak tersip dengan `priority` lebih besar dari 0, pilih satu secara acak di antara bookmark yang memiliki `priority` tertinggi.
2. Jika tidak ada, pilih satu secara acak di antara seluruh bookmark tidak tersip.
3. Jika tidak ada juga, tampilkan pesan *Belum ada bookmark tersimpan*.

Implementasikan logika tersebut di dalam `show_main` pada `main/views.py`, bukan di `template`. Uji keempat keadaan berikut secara manual dan pastikan tidak ada yang menghasilkan galat 500.

- Basis data kosong.
- Seluruh bookmark tersip.
- Seluruh bookmark tidak tersip dengan `priority` bernilai 0.
- Terdapat beberapa bookmark dengan `priority` tertinggi yang sama, sehingga hasilnya harus berubah-ubah saat halaman dimuat ulang berkali-kali.

8 Serialisasi JSON dan XML

8.1 Serializer bawaan Django

```
from django.core import serializers
from django.http import HttpResponse

def notes_data(request):
    notes = Note.objects.all()
    if request.GET.get("format") == "xml":
        payload = serializers.serialize("xml", notes)
        return HttpResponse(payload, content_type="application/xml")

    payload = serializers.serialize("json", notes)
    return HttpResponse(payload, content_type="application/json")
```

Keluaran `serializers.serialize` berbentuk daftar objek dengan kunci `model`, `pk`, dan `fields`. Bentuk tersebut memang bertele-tele, tetapi merupakan format baku Django dan biasanya itulah yang dimaksud ketika sebuah spesifikasi menyebut serialisasi bawaan.

8.2 Header Content-Type bukan sekadar pelengkap

Argumen `content_type` bukan hiasan. Nilai tersebut memberi tahu klien cara menafsirkan isi respons, dan pengujian otomatis lazim memeriksanya secara langsung. Tanpa argumen tersebut Django memakai `text/html`, sehingga klien yang mengandalkan header akan menolak respons Anda walaupun isi datanya sudah benar.

Format	Nilai Content-Type
JSON	application/json
XML	application/xml
HTML	text/html

8.3 JsonResponse untuk struktur buatan sendiri

Bila spesifikasi meminta bentuk JSON yang lebih ringkas daripada format baku Django, susun sendiri daftar kamusnya lalu pakai `JsonResponse`. Kelas tersebut mengatur `Content-Type` secara otomatis.

```
from django.http import JsonResponse

def notes_data(request):
    payload = [
        {
            "id": str(note.id),
            "title": note.title,
            "is_pinned": note.is_pinned,
            "created_at": note.created_at.isoformat(),
        }
        for note in Note.objects.all()
    ]
    return JsonResponse(payload, safe=False)
```

Argumen `safe=False` diperlukan karena `JsonResponse` secara bawaan menolak objek tingkat atas selain kamus. Perhatikan pula bahwa `UUID` dan `datetime` tidak dapat diserialisasi langsung sehingga keduanya dikonversi memakai `str()` dan `isoformat()`.

8.4 Menyaring lewat query parameter

```
def notes_data(request):
    notes = Note.objects.all()

    category = request.GET.get("category")
    if category:
        notes = notes.filter(category=category)

    fmt = request.GET.get("format", "json")
    ...
```

Method `request.GET.get(kunci, cadangan)` adalah cara baku membaca query parameter. Method tersebut mengembalikan nilai cadangan bila parameter tidak dikirim sehingga tidak perlu memeriksa keberadaan kunci lebih dahulu.

8.5 Drill 7: Endpoint Serialisasi

Drill 7 (20 menit)

Implementasikan `/bookmarks/data/` yang hanya menerima GET dengan ketentuan berikut.

- Tanpa query parameter, respons berupa JSON dengan header `Content-Type: application/json`.
- Dengan `?format=xml`, respons berupa XML dengan header `Content-Type: application/xml`.

- Dengan `?archived=true` atau `?archived=false`, hasil disaring berdasarkan `is_archived`. Parameter ini harus dapat digabungkan dengan `format`.
- Format yang tidak dikenal, misalnya `?format=yaml`, dijawab dengan status 400 dan bukan dengan galat 500.

Verifikasi memakai `curl -i` atau tab Network pada peramban, dan pastikan Anda membaca baris header responsnya, bukan hanya isinya.

9 Pengujian Otomatis

Pengujian adalah bagian yang paling sering dikorbankan ketika waktu menipis, padahal justru bagian yang paling mudah diprediksi. Polanya berulang dan dapat ditulis cepat begitu Anda hafal strukturnya.

9.1 Struktur berkas uji

Django menemukan pengujian pada berkas yang namanya diawali `test`. Bila spesifikasi meminta berkas di dalam subdirektori seperti `main/tests/test_fitur.py`, pastikan direktori `main/tests/` berisi `__init__.py` yang kosong. Tanpa berkas tersebut, pengujian Anda tidak akan ditemukan dan cakupan Anda tercatat nol meskipun berkas ujinya ada.

```
main/
  tests/
    __init__.py      # WAJIB, boleh kosong
    test_notes.py
```

9.2 Kerangka pengujian

```
from django.test import TestCase
from django.urls import reverse

from main.models import Note

class NoteModelTest(TestCase):
    def test_defaults_are_applied_on_create(self):
        note = Note.objects.create(title="Rapat", body="Pukul sembilan")
        self.assertFalse(note.is_pinned)
        self.assertEqual(note.view_count, 0)
        self.assertIsNotNone(note.created_at)
        self.assertEqual(str(note), "Rapat")

    def test_ordering_is_newest_first(self):
        older = Note.objects.create(title="Lama", body="x")
        newer = Note.objects.create(title="Baru", body="y")
        self.assertEqual(list(Note.objects.all()), [newer, older])

class NoteViewTest(TestCase):
    def setUp(self):
        self.list_url = reverse("main:show_notes")

    def test_page_renders_with_correct_template(self):
        response = self.client.get(self.list_url)
        self.assertEqual(response.status_code, 200)
```

```

        self.assertTemplateUsed(response, "notes.html")

    def test_empty_state_message_is_shown(self):
        response = self.client.get(self.list_url)
        self.assertContains(response, "Belum ada catatan.")

    def test_post_creates_note_and_redirects(self):
        response = self.client.post(
            self.list_url, {"title": "Judul", "body": "Isi", "category": "general"}
        )
        self.assertRedirects(response, self.list_url)
        self.assertEqual(Note.objects.count(), 1)

    def test_invalid_post_does_not_create_note(self):
        response = self.client.post(self.list_url, {"title": "", "body": ""})
        self.assertEqual(Note.objects.count(), 0)
        self.assertEqual(response.status_code, 200)

    def test_toggle_flips_pin_state(self):
        note = Note.objects.create(title="A", body="B")
        url = reverse("main:toggle_pinned_note", args=[note.id])

        self.client.post(url)
        note.refresh_from_db()
        self.assertTrue(note.is_pinned)

        self.client.post(url)
        note.refresh_from_db()
        self.assertFalse(note.is_pinned)

    def test_toggle_rejects_get(self):
        note = Note.objects.create(title="A", body="B")
        url = reverse("main:toggle_pinned_note", args=[note.id])
        self.assertEqual(self.client.get(url).status_code, 405)

    def test_delete_removes_note(self):
        note = Note.objects.create(title="A", body="B")
        url = reverse("main:delete_note", args=[note.id])

        response = self.client.post(url)
        self.assertRedirects(response, self.list_url)
        self.assertFalse(Note.objects.filter(pk=note.id).exists())

class NoteDataEndpointTest(TestCase):
    def setUp(self):
        Note.objects.create(title="Satu", body="Isi satu")
        self.url = reverse("main:notes_data")

    def test_json_is_the_default_format(self):
        response = self.client.get(self.url)
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response["Content-Type"], "application/json")
        self.assertEqual(len(response.json()), 1)

    def test_xml_format_is_served_when_requested(self):
        response = self.client.get(self.url, {"format": "xml"})
        self.assertEqual(response["Content-Type"], "application/xml")
        self.assertIn(b"<?xml", response.content)

```

9.3 Asersi yang perlu dihafal

Asersi	Memeriksa
<code>assertEqual(a, b)</code>	Kesamaan nilai
<code>assertTrue(x), assertFalse(x)</code>	Nilai kebenaran
<code>assertIsNone(x), assertIsNotNone(x)</code>	Keberadaan nilai
<code>assertContains(resp, teks)</code>	Status 200 sekaligus keberadaan teks pada isi
<code>assertNotContains(resp, teks)</code>	Ketiadaan teks pada isi
<code>assertRedirects(resp, url)</code>	Status 302 sekaligus tujuan redirect
<code>assertTemplateUsed(resp, nama)</code>	Template yang dipakai
<code>assertQuerySetEqual(qs, daftar)</code>	Isi QuerySet

refresh_from_db

Objek Python yang Anda pegang di dalam pengujian tidak otomatis mengetahui perubahan yang dilakukan view pada basis data. Setelah memanggil `self.client.post(...)`, panggil `objek.refresh_from_db()` sebelum memeriksa atributnya. Kelalaian tersebut adalah penyebab paling umum dari pengujian yang gagal padahal kode aplikasinya sudah benar.

9.4 Basis data uji bersifat terisolasi

Django membuat basis data terpisah untuk pengujian dan mengosongkannya kembali setelah setiap method uji selesai. Anda tidak perlu menghapus data secara manual, dan data pada basis data pengembangan Anda tidak terpengaruh. Setiap method uji karena itu harus membuat sendiri data yang dibutuhkan, biasanya melalui `setUp`.

9.5 Mengukur cakupan

```
coverage run manage.py test
coverage report
coverage report -m          # menampilkan nomor baris yang belum teruji
coverage html               # laporan HTML di htmlcov/index.html
```

Kolom Missing pada keluaran `coverage report -m` adalah panduan paling efisien untuk menaikkan angka cakupan. Kolom tersebut menunjukkan baris mana yang belum pernah dieksekusi sehingga Anda dapat menulis pengujian yang menyasar tepat ke baris itu. Baris yang biasanya tertinggal adalah cabang `else`, penanganan form tidak valid, dan cabang penanganan keadaan kosong.

Menaikkan cakupan secara cepat

Setiap cabang `if` memerlukan dua pengujian, satu untuk kondisi terpenuhi dan satu untuk kondisi tidak terpenuhi. Menulis pasangan pengujian untuk setiap percabangan pada `views.py` biasanya cukup untuk melampaui ambang 85 persen, karena `views.py` adalah berkas dengan jumlah baris logika terbanyak.

```
# .coveragerc, opsional, mengeluarkan berkas yang tidak perlu diukur
[run]
omit =
    */migrations/*
    */tests/*
    manage.py
    */settings.py
```

9.6 Drill 8: Menulis Pengujian dan Menaikkan Cakupan

Drill 8 (25 menit)

Direktori `main/tests/` pada kerangka sudah memiliki `__init__.py` beserta `test_smoke.py` sebagai contoh. Tambahkan `main/tests/test_bookmarks.py` berisi pengujian yang mencakup keseluruhan perilaku berikut.

- Nilai bawaan field dan pengurutan bawaan pada model.
- Halaman daftar merender status 200 dan memakai template yang benar.
- Pesan keadaan kosong muncul saat basis data kosong.
- POST yang valid menambah satu baris dan menghasilkan redirect.
- POST yang tidak valid tidak menambah baris dan mengembalikan status 200.
- Toggle arsip membalik nilai dan menolak GET dengan status 405.
- Penghapusan menghilangkan baris dan menolak GET dengan status 405.
- Id yang tidak ada menghasilkan 404 pada rute arsip dan rute hapus.
- Beranda menampilkan pesan cadangan saat seluruh bookmark terarsip.
- Endpoint data mengembalikan JSON dan XML dengan header masing-masing.
- Format tidak dikenal mengembalikan status 400.

Kemudian jalankan `coverage run manage.py test` dilanjutkan `coverage report -m`. Catat angka awal, baca kolom `Missing`, lalu tambahkan pengujian sampai cakupan melampaui 90 persen. Tuliskan baris mana saja yang semula terlewat dan mengapa.

10 Kualitas Kode

10.1 Menjalankan ruff

```
ruff check          # menampilkan pelanggaran
ruff check --fix    # memperbaiki yang dapat diperbaiki otomatis
ruff format         # merapikan format kode
```

Jalankan `ruff check --fix` sebelum commit terakhir. Perintah tersebut menghapus impor yang tidak terpakai dan merapikan format dalam hitungan detik, dua hal yang paling sering tertinggal ketika Anda bekerja cepat.

Kode	Arti	Perbaikan
F401	Impor tidak terpakai	Hapus baris impor tersebut
F841	Variabel lokal tidak terpakai	Hapus variabel atau gunakan hasilnya
E501	Baris terlalu panjang	Pecah baris atau gunakan kurung
E711,	Perbandingan dengan	Tulis <code>is None</code> atau langsung <code>if x</code>
E712	<code>None</code> atau boolean	
E722	<code>except</code> tanpa tipe	Sebutkan tipe <i>exception</i> yang ditangani

10.2 Kebiasaan yang menjaga kualitas kode

- Menulis nama yang bermakna. Nama `visible_notes` memberi informasi yang tidak diberikan data atau `x`.
- Menghapus `print`, kode yang dikomentari, dan fungsi yang tidak lagi dipanggil sebelum commit terakhir.

- Memakai HTML semantik seperti `<main>`, `<article>`, `<nav>`, `<section>`, dan `<time>`, bukan `<div>` untuk segalanya.
- Menyertakan atribut aksesibilitas dasar, misalnya `<label for>` pada setiap input dan `alt` pada setiap gambar.
- Menaruh logika kueri di view atau model, bukan di template.
- Menulis komentar hanya untuk menjelaskan alasan sebuah keputusan, bukan mengulang apa yang sudah terbaca dari kodenya.

10.3 Drill 9: Menemukan Cacat pada Kode Orang Lain

Drill berikut melatih kemampuan yang paling menentukan pada kondisi bertekanan waktu, yaitu membaca kode dan menemukan cacatnya tanpa menjalankannya.

Drill 9 (20 menit)

Potongan kode berikut memuat sedikitnya sepuluh cacat. Temukan sebanyak mungkin, klasifikasikan masing-masing sebagai masalah keamanan, kebenaran, kualitas kode, atau pelanggaran spesifikasi, lalu tuliskan perbaikannya.

```
import uuid, random, json
from django.shortcuts import render, redirect
from django.http import HttpResponse
from django.core import serializers
from main.models import Bookmark

def show_bookmarks(request):
    if request.method == "POST":
        b = Bookmark(label=request.POST["label"], url=request.POST["url"])
        b.save()
        return render(request, "bookmarks.html", {"bookmarks": Bookmark.objects.all()})
    html = "<ul>"
    for b in Bookmark.objects.all():
        html += "<li>" + b.label + "</li>"
    html += "</ul>"
    print("rendering", html)
    return HttpResponse(html)

def toggle_archive(request, id):
    b = Bookmark.objects.get(id=id)
    b.is_archived = not b.is_archived
    b.save()
    return redirect("/bookmarks/")

def delete_bookmark(request, id):
    Bookmark.objects.get(id=id).delete()
    return redirect("/bookmarks/")

def show_main(request):
    items = Bookmark.objects.filter(is_archived=False)
    featured = items[0]
    return render(request, "home.html", {"featured": featured})

def bookmarks_data(request):
    if request.GET["format"] == "xml":
        return HttpResponse(serializers.serialize("xml", Bookmark.objects.all()))
```

```
return HttpResponse(serializers.serialize("json", Bookmark.objects.all()))
```

11 Daftar Kesalahan yang Paling Sering Terjadi

Gejala	Penyebab	tersering	Perbaikan
404 pada rute yang baru dibuat	Rute belum didaftarkan atau garis miring penutup berbeda		Periksa <code>urls.py</code> aplikasi dan <code>include</code> pada <code>urls.py</code> proyek
404 pada rute ber-UUID	Konverter <code>int</code> dipakai untuk primary key UUID		Ganti menjadi <code><uuid:...></code>
403 CSRF verification failed	<code>{% csrf_token %}</code> tidak ada pada form POST		Tambahkan tag tersebut di dalam setiap <code><form method="post"></code>
405 saat tombol ditekan	Tombol dibuat sebagai tautan GET		Bungkus tombol dengan form POST
500 dengan <code>DoesNotExist</code>	Memakai <code>objects.get</code> untuk id yang mungkin tidak ada		Ganti dengan <code>get_object_or_404</code>
500 dengan <code>IndexError</code>	Mengakses <code>qs[0]</code> pada <code>QuerySet</code> kosong		Gunakan <code>.first()</code> lalu periksa <code>None</code>
Data tidak muncul di halaman	Nama kunci context berbeda dari nama variabel template		Samakan ejaan keduanya
Data tersimpan ganda saat muat ulang	View merender langsung dari POST		Terapkan Post/Redirect/Get
<code>no such table</code>	Migrasi belum dijalankan		Jalankan <code>makemigrations</code> lalu <code>migrate</code>
Pengujian tidak ditemukan	<code>__init__.py</code> tidak ada di direktori tests		Buat berkas kosong tersebut
Cakupan tercatat nol	Pengujian gagal dijalankan sejak awal		Baca pesan galat dari <code>coverage run</code>

12 Latihan Terpadu: 100 Menit

Latihan Terpadu: Papan Kutipan Harian

Kerjakan dengan timer 100 menit, tanpa membuka catatan, tanpa bantuan alat pembangkit kode, dan hanya dengan dokumentasi Django resmi yang dibuka di satu tab. Batasan tersebut adalah bagian dari latihannya.

Titik awal. Kloning ulang repositori kerangka ke direktori baru agar Anda benar-benar mulai dari nol, bukan melanjutkan pekerjaan drill.

```
git clone https://github.com/aldofahrezy/latihan-quiz-1.git terpadu
cd terpadu
python -m venv env && source env/bin/activate
pip install -r requirements.txt && python manage.py migrate
```

Latar. Situs tersebut akan menampilkan koleksi kutipan harian. Pengelola harus dapat menambah, menerbitkan, menyukai, dan menghapus kutipan, sedangkan pengunjung melihat satu kutipan pilihan pada halaman beranda.

1. Model. Buat model `Quote` pada aplikasi main dengan field `id` berupa UUID sebagai primary key, `text` bertipe teks panjang, `author` bertipe teks pendek maksimal 80 karakter yang boleh kosong, `mood` bertipe pilihan dengan tiga nilai yaitu `calm`, `bold`, dan `funny`, `is_draft` bertipe boolean bernilai bawaan `True`, `likes` bertipe bilangan bulat tak negatif bernilai bawaan 0, serta `created_at` yang terisi otomatis saat pembuatan. Urutan bawaan adalah dari yang terbaru. Buat dan terapkan migrasinya.

2. Halaman pengelolaan. Sediakan halaman pada rute `/quotes/` dengan nama rute `show_quotes`. Halaman tersebut menampilkan seluruh kutipan, menampilkan pesan *Belum ada kutipan.* ketika kosong, serta menyediakan form penambahan yang dikirim lewat `POST /quotes/` dengan token CSRF dan validasi. Setelah penambahan berhasil, lakukan redirect kembali ke `/quotes/`.

3. Aksi per baris. Setiap kutipan memiliki tiga tombol. Tombol publikasi mengirim `POST /quotes/<id>/publish/` dengan nama rute `toggle_draft_quote`, membalik nilai `is_draft`, dan teksnya berbunyi *Terbitkan* ketika kutipan masih berstatus draf serta *Jadikan Draf* ketika sudah terbit. Tombol suka mengirim `POST /quotes/<id>/like/` dengan nama rute `like_quote` dan menaikkan `likes` sebanyak satu. Tombol hapus mengirim `POST /quotes/<id>/delete/` dengan nama rute `delete_quote`. Ketiga rute menolak method selain `POST` dan melakukan redirect kembali ke `/quotes/`.

4. Kutipan pilihan di beranda. Pada halaman `/` yang dirender lewat `main/home.html`, tampilkan satu kutipan pilihan dengan aturan berikut. Pilih secara acak di antara kutipan yang sudah terbit dan memiliki jumlah suka terbanyak. Jika tidak ada kutipan terbit sama sekali, tampilkan pesan *Belum ada kutipan.* tanpa menimbulkan galat server.

5. Endpoint data. Sediakan `/quotes/data/` dengan nama rute `quotes_data` yang hanya menerima `GET`. Secara bawaan endpoint mengembalikan JSON dengan header `application/json`. Dengan `?format=xml` endpoint mengembalikan XML dengan header `application/xml`. Dengan `?mood=calm` hasil disaring berdasarkan nilai `mood`, dan parameter tersebut dapat digabungkan dengan `format`.

6. Pengujian. Tulis pengujian pada `main/tests/test_quotes.py` yang mencakup model, seluruh view pengelolaan, kutipan pilihan beserta keadaan kosongnya, dan endpoint data beserta headernya. Targetkan cakupan minimal 85 persen dan catat hasil `coverage report`.

7. Kualitas. Pastikan `ruff check` bersih, struktur MTV terpisah dengan benar, dan HTML memakai elemen semantik.

8. Riwayat perubahan. Buat commit setiap kali satu bagian selesai berfungsi. Di akhir sesi, pastikan git status bersih dan seluruh berkas, termasuk berkas migrasi, sudah masuk ke dalam riwayat.

Cara mengevaluasi hasil latihan Anda

Setelah 100 menit berakhir, berhentilah walaupun belum selesai. Telusuri daftar periksa mandiri pada kunci jawaban, lalu catat dua hal. Bagian mana yang tidak sempat dikerjakan, dan pada menit ke berapa Anda mulai tertinggal dari perkiraan Anda sendiri. Kedua catatan tersebut jauh lebih berguna daripada jumlah butir yang berhasil diselesaikan, karena keduanya menunjukkan bagian mana yang perlu dilatih ulang.

13 Soal Konseptual

Jawab secara ringkas, satu sampai tiga kalimat per nomor. Pertanyaan berikut menguji alasan di balik pola yang Anda pakai, dan pemahaman itulah yang membedakan kode yang kebetulan berjalan dari kode yang benar.

1. Jelaskan mengapa operasi penghapusan tidak boleh dapat dipicu lewat permintaan GET, dan sebutkan satu skenario konkret ketika penghapusan berbasis GET menghilangkan data tanpa ada pengguna yang menekan tombol apa pun.
2. Apa perbedaan hasil antara `default=uuid.uuid4` dan `default=uuid.uuid4()` pada definisi field, dan mengapa bentuk kedua menimbulkan kegagalan?
3. Sebuah view merender langsung halaman daftar setelah memproses POST tanpa melakukan redirect. Semua pengujian lulus. Jelaskan cacat yang tetap ada beserta cara pengguna menemukannya.
4. Apa perbedaan antara `auto_now_add=True` dan `auto_now=True`, dan mengapa keduanya tidak muncul pada `ModelForm`?
5. Sebutkan kode status HTTP yang tepat untuk masing-masing keadaan berikut beserta alasannya: id tidak ditemukan, method tidak diizinkan, token CSRF tidak valid, dan penambahan data berhasil pada pola Post/Redirect/Get.
6. Cakupan pengujian sebuah proyek tercatat 95 persen, tetapi sebuah bug lolos ke produksi. Jelaskan mengapa angka cakupan yang tinggi tidak menjamin kebenaran.
7. Mengapa pemanggilan objek `objek.refresh_from_db()` diperlukan setelah `self.client.post(...)` di dalam pengujian?
8. Bandingkan `order_by("?").first()` dengan `random.choice(list(qs))` dari sisi keterbacaan, penggunaan memori, dan biaya pada basis data.
9. Sebuah halaman tidak menampilkan data apa pun, tetapi tidak ada galat yang muncul dan basis data terbukti berisi baris. Sebutkan tiga penyebab yang paling mungkin dan cara memeriksanya secara berurutan.
10. Jelaskan apa yang sebenarnya dilindungi oleh token CSRF, dan mengapa token tersebut tidak diperlukan pada form pencarian yang memakai method GET.

14 Kunci Jawaban dan Pembahasan

Cara memakai bab ini

Buka bab ini hanya setelah Anda benar-benar mencoba mengerjakan soalnya. Nilai latihan terletak pada proses menemukan jawabannya, bukan pada membaca jawaban yang sudah jadi. Untuk soal terpadu, kunci yang diberikan berupa kerangka dan daftar Periksa, bukan kode lengkap, agar Anda tetap menyusun implementasinya sendiri.

14.1 Kunci Drill 1

```
import uuid
from django.db import models

class Bookmark(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    label = models.CharField(max_length=100)
    url = models.URLField()
    priority = models.PositiveIntegerField(default=0)
    is_archived = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    last_opened_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-priority", "-created_at"]

    def __str__(self):
        return self.label
```

Dua hal yang sering salah pada drill ini. Pertama, field tautan sebaiknya memakai `URLField` agar Django memvalidasi formatnya, bukan `CharField` biasa. Kedua, field yang boleh kosong memerlukan dua argumen yang berbeda maknanya. Argumen `null=True` mengizinkan nilai NULL pada basis data, sedangkan `blank=True` mengizinkan field tersebut dikosongkan pada form. Untuk field non-teks, keduanya biasanya dipakai bersamaan.

14.2 Kunci Drill 2

```
from django.urls import path

from main import views

app_name = "main"

urlpatterns = [
    path("", views.show_main, name="show_main"),  # sudah ada di kerangka
    path("bookmarks/", views.show_bookmarks, name="show_bookmarks"),
    path("bookmarks/data/", views.bookmarks_data, name="bookmarks_data"),
    path("bookmarks/<uuid:pk>/archive/", views.toggle_archive, name="toggle_archive"),
    path("bookmarks/<uuid:pk>/delete/", views.delete_bookmark, name="delete_bookmark"),
]
```

Jawaban pertanyaan pertama adalah permintaan ke `/bookmarks/data/` akan ditangani oleh view tag, bukan oleh view data. Penyebabnya adalah Django mencocokkan pola dari atas ke bawah dan berhenti pada kecocokan pertama, sedangkan string `data` memenuhi konverter `str`. Perbaikannya adalah meletakkan pola literal yang lebih spesifik di atas pola bervariasi, seperti pada contoh di atas.

Jawaban pertanyaan kedua adalah sebagai berikut.

```
reverse("main:toggle_archive", args=[b.id])    # Python
{% url 'main:toggle_archive' b.id %}           # template
```

14.3 Kunci Drill 3

```
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from main.models import Bookmark

def show_bookmarks(request):
    return render(request, "main/bookmarks.html", {"bookmarks": Bookmark.objects.all()})

@require_POST
def toggle_archive(request, pk):
    bookmark = get_object_or_404(Bookmark, pk=pk)
    bookmark.is_archived = not bookmark.is_archived
    bookmark.save(update_fields=["is_archived"])
    return redirect("main:show_bookmarks")

@require_POST
def delete_bookmark(request, pk):
    get_object_or_404(Bookmark, pk=pk).delete()
    return redirect("main:show_bookmarks")
```

Tiga keputusan pada kunci di atas layak diperhatikan. Dekorator `@require_POST` yang menghasilkan 405, `get_object_or_404` yang menghasilkan 404 alih-alih 500, serta `redirect` yang memakai nama rute sehingga URL dapat berubah tanpa menyentuh `views.py`.

Argumen `update_fields` pada `save` membatasi kolom yang ditulis ulang ke basis data. Argumen tersebut bersifat opsional, tetapi menyatakan maksud Anda secara eksplisit dan menghindari penimpaan kolom lain yang mungkin berubah bersamaan.

14.4 Kunci Drill 4

```
# main/forms.py
from django import forms

from main.models import Bookmark

class BookmarkForm(forms.ModelForm):
    class Meta:
        model = Bookmark
        fields = ["label", "url", "priority"]

    def clean_label(self):
        label = self.cleaned_data["label"].strip()
        if len(label) < 3:
            raise forms.ValidationError("Label minimal tiga karakter.")
        return label

    def clean_priority(self):
        priority = self.cleaned_data["priority"]
```

```

    if priority > 10:
        raise forms.ValidationError("Prioritas maksimal 10.")
    return priority

```

```

# main/views.py
from main.forms import BookmarkForm

def show_bookmarks(request):
    if request.method == "POST":
        form = BookmarkForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("main:show_bookmarks")
    else:
        form = BookmarkForm()

    context = {"form": form, "bookmarks": Bookmark.objects.all()}
    return render(request, "main/bookmarks.html", context)

```

Perhatikan struktur percabangannya. Redirect hanya terjadi pada jalur POST yang valid. Jalur POST yang tidak valid sengaja jatuh ke `render` di bagian bawah fungsi, sehingga form yang dikembalikan masih membawa masukan pengguna beserta pesan galatnya. Bila jalur tersebut ikut diakhiri `redirect`, pengguna kehilangan seluruh ketikannya dan tidak pernah melihat alasan penolakannya.

Method `clean_<nama field>` dijalankan Django secara otomatis pada tahap validasi dan wajib mengembalikan nilai yang sudah dibersihkan. Lupa menuliskan `return` membuat field tersebut bernilai `None`.

Hasil eksperimen penghapusan token adalah status 403 dengan halaman berjudul Forbidden dan keterangan CSRF verification failed. Request aborted. Perhatikan bahwa halaman tetap dapat dibuka seperti biasa dan galat hanya muncul pada saat pengiriman form. Sifat tersebut menjelaskan mengapa menguji tampilan halaman saja tidak cukup.

14.5 Kunci Drill 5

```

{% extends 'base.html' %}
{% block content %}
<h1>Bookmark</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Tambah</button>
</form>

<ul>
    {% for bookmark in bookmarks %}
    <li>
        <article>
            <h2><a href="{{ bookmark.url }}">{{ bookmark.label }}</a></h2>
            <p>Prioritas: {{ bookmark.priority }}</p>
            <time datetime="{{ bookmark.created_at|date:'c' }}">
                {{ bookmark.created_at|date:"d M Y" }}
            </time>
        </article>

        <form method="post" action="{% url 'main:toggle_archive' bookmark.id %}">
            {% csrf_token %}

```

```

        <button type="submit">
            {% if bookmark.is_archived %}Kembalikan{% else %}Arsipkan{% endif %}
        </button>
    </form>

    <form method="post" action="{% url 'main:delete_bookmark' bookmark.id %}">
        {% csrf_token %}
        <button type="submit">Hapus</button>
    </form>
</li>
{% empty %}
<li>Belum ada bookmark tersimpan.</li>
{% endfor %}
</ul>
{% endblock %}

```

Salah tulis nama variabel tidak menimbulkan galat karena Django Template Language dirancang agar kegagalan resolusi variabel menghasilkan string kosong. Perilaku tersebut berbahaya justru karena tidak menghasilkan sinyal apa pun. Halaman terlihat berhasil dirender, pengujian manual yang tergesa menganggapnya benar, dan cacatnya baru terlihat ketika seseorang membandingkan isi halaman dengan isi basis data. Karena itu asersi `assertContains` yang memeriksa nilai data sesungguhnya jauh lebih berguna daripada sekadar memeriksa status 200.

14.6 Kunci Drill 6

```

from django.db.models import Max
from django.shortcuts import render

from main.models import Bookmark

def show_main(request):
    active = Bookmark.objects.filter(is_archived=False)

    top_priority = active.aggregate(Max("priority"))["priority__max"]
    if top_priority:
        candidates = active.filter(priority=top_priority)
    else:
        candidates = active

    # order_by("?") mengacak di sisi basis data; memadai untuk tabel kecil.
    featured = candidates.order_by("?").first()

    return render(request, "main/home.html", {"featured": featured})

```

Nilai `top_priority` bernilai `None` ketika tidak ada baris sama sekali dan bernilai 0 ketika seluruh baris memiliki prioritas nol. Pemeriksaan `if top_priority` menangani kedua keadaan tersebut sekaligus, karena `None` dan 0 sama-sama bernilai salah. Pemanggilan `.first()` pada `QuerySet` kosong mengembalikan `None` sehingga template hanya perlu satu percabangan.

```

{% if featured %}
    <blockquote>{{ featured.label }}</blockquote>
{% else %}
    <p>Belum ada bookmark tersimpan.</p>
{% endif %}

```

14.7 Kunci Drill 7

```

from django.core import serializers
from django.http import HttpResponse, HttpResponseBadRequest
from django.views.decorators.http import require_GET

from main.models import Bookmark

CONTENT_TYPES = {"json": "application/json", "xml": "application/xml"}

@require_GET
def bookmarks_data(request):
    fmt = request.GET.get("format", "json")
    if fmt not in CONTENT_TYPES:
        return HttpResponseBadRequest("Format tidak didukung. Gunakan json atau xml.")

    bookmarks = Bookmark.objects.all()

    archived = request.GET.get("archived")
    if archived in {"true", "false"}:
        bookmarks = bookmarks.filter(is_archived=(archived == "true"))

    payload = serializers.serialize(fmt, bookmarks)
    return HttpResponse(payload, content_type=CONTENT_TYPES[fmt])

```

Pemetaan `CONTENT_TYPES` melayani dua kebutuhan sekaligus. Kamus tersebut menjadi daftar format yang diizinkan sekaligus sumber nilai header, sehingga tidak ada kemungkinan format valid yang terkirim dengan header salah. Perhatikan pula bahwa `request.GET.get("format", "json")` memakai nilai cadangan, berbeda dari `request.GET["format"]` yang melemparkan `MultiValueDictKeyError` ketika parameter tidak dikirim.

14.8 Kunci Drill 8

Kerangka pengujiannya mengikuti pola pada Bab 9. Empat pengujian berikut adalah yang paling sering terlewat sehingga layak diperhatikan khusus.

```

def test_archive_rejects_get(self):
    bookmark = Bookmark.objects.create(label="Situs", url="https://a.test")
    url = reverse("main:toggle_archive", args=[bookmark.id])
    self.assertEqual(self.client.get(url).status_code, 405)

def test_missing_id_returns_404(self):
    url = reverse("main:delete_bookmark", args=[uuid.uuid4()])
    self.assertEqual(self.client.post(url).status_code, 404)

def test_home_falls_back_when_everything_is_archived(self):
    Bookmark.objects.create(label="Situs", url="https://a.test", is_archived=True)
    response = self.client.get(reverse("main:show_main"))
    self.assertEqual(response.status_code, 200)
    self.assertContains(response, "Belum ada bookmark tersimpan.")

def test_unknown_format_returns_400(self):
    response = self.client.get(reverse("main:bookmarks_data"), {"format": "yaml"})
    self.assertEqual(response.status_code, 400)

```

Baris yang biasanya masih terlewat setelah putaran pertama adalah cabang `else` pada percabangan method, jalur form tidak valid, jalur penanganan format yang tidak dikenal, dan cabang keadaan kosong pada pemilihan data sorotan. Keempatnya memiliki ciri yang sama, yaitu meru-

pakan jalur yang tidak pernah dilalui pada pemakaian normal. Karena itu jalur tersebut juga merupakan tempat bug paling sering bersembunyi.

14.9 Kunci Drill 9

No	Cacat	Kategori	Perbaikan
1	<code>toggle_archive</code> dan <code>delete_bookmark</code> menerima GET	Keamanan	Tambahkan <code>@require_POST</code> pada kedua view
2	<code>objects.get</code> untuk id yang mungkin tidak ada	Kebenaran	Ganti dengan <code>get_object_or_404</code>
3	<code>request.POST["label"]</code> tanpa nilai cadangan	Kebenaran	Gunakan <code>.get()</code> atau <code>ModelForm</code>
4	Tidak ada validasi masukan sama sekali	Kebenaran	Validasi lewat <code>ModelForm</code>
5	POST valid diakhiri <code>render</code> , bukan <code>redirect</code>	Kebenaran	Terapkan <code>Post/Redirect/Get</code>
6	HTML disusun manual di dalam view	Kualitas	Pindahkan ke template dan panggil <code>render</code>
7	Penggabungan string tanpa <i>escaping</i>	Keamanan	Template Django melakukan <i>escaping</i> otomatis
8	<code>print</code> tertinggal di kode produksi	Kualitas	Hapus
9	<code>items[0]</code> pada QuerySet yang mungkin kosong	Kebenaran	Gunakan <code>.first()</code> lalu tangani <code>None</code>
10	<code>request.GET["format"]</code> tanpa nilai cadangan	Kebenaran	Gunakan <code>.get("format", "json")</code>
11	<code>content_type</code> tidak diatur pada endpoint data	Spesifikasi	Tambahkan <code>application/json</code> dan <code>application/xml</code>
12	Impor <code>uuid</code> , <code>random</code> , dan <code>json</code> tidak terpakai	Kualitas	Hapus, atau jalankan <code>ruff check --fix</code>
13	Beberapa impor pada satu baris	Kualitas	Pisahkan sesuai PEP 8
14	<code>Redirect</code> memakai URL literal	Kualitas	Gunakan nama rute
15	Parameter bernama <code>id</code> menutupi <i>builtin</i>	Kualitas	Ganti menjadi <code>pk</code> atau <code>bookmark_id</code>

Cacat nomor 7 layak diperhatikan lebih jauh karena merupakan celah keamanan yang nyata. Label yang berisi `<script>alert(1)</script>` akan tereksekusi di peramban pengunjung karena string HTML disusun manual tanpa *escaping*. Template Django melakukan *escaping* secara bawaan, dan hal tersebut merupakan salah satu alasan utama mengapa penyusunan HTML di dalam view tidak pernah dapat dibenarkan.

14.10 Kunci Soal Terpadu

Kunci berikut berbentuk kerangka dan daftar periksa. Bagian yang paling menentukan pada soal terpadu adalah pemilihan kutipan pilihan, karena bagian tersebut menggabungkan agregasi, penyaringan, pengacakan, dan penanganan keadaan kosong sekaligus.

```
from django.db.models import Max

def show_main(request):
    published = Quote.objects.filter(is_draft=False)
```

```

most_likes = published.aggregate(Max("likes"))["likes__max"]

featured = None
if most_likes is not None:
    featured = published.filter(likes=most_likes).order_by("?").first()

return render(request, "main/home.html", {"featured": featured})

```

Perhatikan bahwa pemeriksaannya memakai `is not None`, bukan sekadar `if most_likes`. Perbedaan tersebut penting pada soal ini. Nilai 0 adalah jumlah suka yang sah untuk kutipan yang sudah terbit, sedangkan `None` berarti tidak ada kutipan terbit sama sekali. Menulis `if most_likes` akan menganggap kumpulan kutipan yang seluruhnya bernilai nol suka sebagai kumpulan kosong, dan halaman akan salah menampilkan pesan cadangan. Kasus semacam itulah yang membedakan implementasi yang memenuhi spesifikasi dari implementasi yang hanya tampak benar.

Daftar periksa mandiri

Bagian	Terpenuhi jika
Model dan migrasi	Tujuh field sesuai tipe yang diminta, mood memakai choices, Meta.ordering benar, migrasi diterapkan tanpa galat
Halaman pengelolaan	Daftar tampil, pesan kosong muncul, form POST bervalidasi dan ber-CSRF, redirect setelah berhasil
Aksi per baris	Ketiga rute menolak non-POST dengan 405, teks tombol publikasi berubah sesuai status, likes bertambah tepat satu
Kutipan pilihan	Acak di antara yang suka terbanyak, pesan cadangan tampil tanpa galat 500, kasus nol suka tertangani
Endpoint data	JSON bawaan, XML lewat parameter, header benar untuk keduanya, penyaringan mood dapat digabungkan
Pengujian	Keempat area tercakup, seluruh pengujian lulus, cakupan minimal 85 persen
Kualitas kode	ruff check bersih, MTV terpisah, HTML semantik, tanpa kode mati
Riwayat perubahan	git status bersih dan seluruh berkas, termasuk migrasi, sudah di-commit

14.11 Kunci Soal Konseptual

1. GET bersifat aman dan idempoten sehingga peramban, *prefetcher*, *crawler*, dan pemindai tautan boleh memanggilnya tanpa izin pengguna. Skenario konkretnya adalah peramban yang melakukan *prefetch* pada tautan yang terlihat di halaman, atau perayap mesin pencari yang mengunjungi setiap tautan. Keduanya akan menghapus seluruh data tanpa ada yang pernah menekan tombol hapus.
2. Bentuk `uuid.uuid4` menyerahkan fungsinya kepada Django sehingga fungsi tersebut dipanggil ulang setiap kali objek baru dibuat dan setiap baris memperoleh UUID berbeda. Bentuk `uuid.uuid4()` memanggil fungsi tersebut satu kali saat kelas didefinisikan sehingga nilai yang sama menjadi default untuk seluruh baris. Karena field tersebut merupakan primary key, penyimpanan baris kedua melanggar batasan keunikan dan gagal.
3. Cacatnya adalah pengiriman ulang form. Karena respons dihasilkan langsung dari POST, menekan tombol muat ulang membuat peramban menampilkan dialog konfirmasi pengiriman ulang dan mengirim POST yang sama untuk kedua kalinya. Data tersimpan ganda. Pengujian tidak menangkapnya karena `self.client.post` tidak mensimulasikan perilaku muat ulang peramban.

4. Argumen `auto_now_add=True` mengisi nilai hanya pada penyimpanan pertama, sedangkan `auto_now=True` memperbaruinya pada setiap pemanggilan `save()`. Keduanya menjadikan field bersifat non-editable, dan Django memang tidak menyertakan field non-editable pada `ModelForm` karena nilainya dikendalikan sistem, bukan pengguna.
5. Id tidak ditemukan dijawab 404 karena sumber dayanya memang tidak ada. Method tidak diizinkan dijawab 405 karena rutenya ada tetapi method tersebut tidak didukung. Token CSRF tidak valid dijawab 403 karena permintaannya dipahami tetapi ditolak. Penambahan berhasil pada pola Post/Redirect/Get dijawab 302 yang mengarahkan peramban ke halaman hasil.
6. Cakupan hanya mengukur baris mana yang dieksekusi, bukan apakah perilakunya benar. Pengujian dapat menjalankan seluruh baris tanpa pernah memeriksa hasilnya, dan cakupan tetap tinggi walaupun asersinya lemah. Selain itu cakupan baris tidak mengukur cakupan cabang atau kombinasi masukan sehingga bug pada kasus tepi tetap dapat lolos.
7. Objek Python yang Anda pegang menyimpan salinan nilai dari saat objek tersebut diambil. View mengubah baris di basis data, bukan objek Python yang ada di memori pengujian. Pemanggilan `refresh_from_db()` membaca ulang nilai terbaru dari basis data sehingga asersi memeriksa keadaan yang sebenarnya.
8. Bentuk `order_by("?").first()` paling ringkas dan hemat memori karena hanya satu baris yang dibawa ke Python, tetapi memaksa basis data mengacak seluruh tabel sehingga mahal pada tabel besar. Bentuk `random.choice(list(qs))` lebih eksplisit dan lebih mudah dibaca, tetapi memuat seluruh hasil ke memori. Untuk tabel besar, kombinasi `count()` dengan pengindeksan acak adalah kompromi terbaik.
9. Tiga penyebab yang paling mungkin, berurutan dari yang paling murah diperiksa, adalah nama kunci `context` pada view berbeda dari nama variabel pada template, template yang dirender bukan template yang sedang Anda sunting, serta kueri yang tersaring habis oleh filter. Periksa dengan mencetak isi `context` di view, memeriksa nama berkas pada pemanggilan `render`, lalu menjalankan kueri yang sama di `manage.py shell`.
10. Token CSRF melindungi dari permintaan yang diciptakan situs lain memakai kredensial pengguna yang sedang aktif, misalnya `cookie` sesi yang otomatis ikut terkirim. Token tersebut membuktikan bahwa permintaan berasal dari halaman yang benar-benar dilayani oleh aplikasi Anda. Form pencarian berbasis GET tidak memerlukannya karena GET tidak mengubah keadaan sehingga permintaan palsu tidak menimbulkan kerugian.